

LuxeFlats

Flat Rental Microservices Platform

TECHNICAL ARCHITECTURE & IMPLEMENTATION GUIDE

A Comprehensive Beginner-Friendly Guide covering:

- System Architecture & API Gateway Routing
- Microservice Databases & MySQL Schema
 - Security & JWT Authentication Flow
 - End-to-End Booking & Stay Lifecycle
- React Frontend Integration (Axios Interceptors)
- System Design Mock Interview Q&As

Target Audience: Backend Developers, System Architects, & Interview Candidates

Java 21 / Spring Boot 3.x / React Vite / MySQL Database

Generated: August 2026

Table of Contents

1. System Overview & Core Microservices Architecture	3
2. Database Topology & Entity Schema Design	4
3. Security & JWT Authentication Protocol	5
4. Core Backend Microservices & Key Classes	6
5. End-to-End Booking Request Lifecycle	7
6. Payment Integration & stay Completion Sync	8
7. React Frontend Architecture & Axios Interceptors	9
8. Dynamic Owner Earnings & Dashboard Mechanics	10
9. Production Seeding Data & Indian Cities Setup	11
10. System Design & Technical Interview Questions	12

1. System Overview & Core Microservices Architecture

LuxeFlats is a state-of-the-art flat rental marketplace application built using a microservices architecture. The backend consists of five distinct Spring Boot microservices, each running as an isolated process on a dedicated port. Downstream communication is coordinated through a central API Gateway.

Core Microservice Directory Structure:

1. API Gateway (Port 8080): Acts as the single entry point for client requests. It performs CORS routing to forward payloads to downstream microservices based on request path mapping.
2. Auth Service (Port 8081): Manages user account registration, login verification, password hashing via Spring Security's BCryptPasswordEncoder, and issues JWT tokens containing claims.
3. Property Service (Port 8082): Maintains listings of available flats across multiple cities. Handles listing registration, searches, filtering, and availability toggles.
4. Booking Service (Port 8083): Tracks leases, stay durations, date checks, and booking statuses.
5. Payment Service (Port 8084): Logs mock transactions, manages invoice states, and tracks payment confirmations.

Key Microservices Integration Concept:

- Database per Service: Each service owns its database instance. Services do not share tables. They interact via network calls using RestTemplate or WebClient, preserving the autonomy of each domain.
- Shared Secret: Downstream security filters decode incoming Authorization headers by validating the token's cryptographic signature against a shared secret defined in the configuration files (jwt.secret).

2. Database Topology & Entity Schema Design

Each service runs on its own database, preventing tight coupling at the data tier. The schemas are configured automatically via Hibernate's `ddl-auto: update` property. The relational structures of key entities include:

A. auth-service database: flat_rental_auth

- Table: users
 - id (BIGINT, Primary Key, Auto Increment)
 - username (VARCHAR(50), Unique, Nullable=false)
 - email (VARCHAR(100), Unique, Nullable=false)
 - password (VARCHAR(255), Hashed via BCrypt)
 - full_name (VARCHAR(100))
 - phone_number (VARCHAR(20))
 - role (VARCHAR(20), values: ROLE_ADMIN, ROLE_OWNER, ROLE_TENANT)
 - created_at (DATETIME)

B. property-service database: flat_rental_property

- Table: properties
 - id (BIGINT, Primary Key, Auto-Increment)
 - owner_id (BIGINT, maps to users.id from auth-service, no foreign key constraint)
 - title (VARCHAR(150), Nullable=false)
 - description (VARCHAR(2000))
 - address (VARCHAR(255), Nullable=false)
 - city (VARCHAR(100), Nullable=false)
 - rent_amount (DECIMAL(10, 2), Nullable=false)
 - security_deposit (DECIMAL(10, 2))
 - bedrooms (INT), bathrooms (INT), area (INT)
 - available (BOOLEAN, default=true)
 - locality (VARCHAR(150)), furnishing (VARCHAR(50))
 - image_urls (VARCHAR(2000), comma-separated list of Unsplash URLs)
 - amenities (VARCHAR(1000), comma-separated values)

C. booking-service database: flat_rental_booking

- Table: bookings
 - id (BIGINT, Primary Key, Auto-Increment)
 - property_id (BIGINT, maps to properties.id)
 - tenant_id (BIGINT, maps to users.id)
 - start_date (DATE), end_date (DATE)
 - status (VARCHAR(20), values: PENDING, CONFIRMED, COMPLETED, CANCELLED)

D. payment-service database: flat_rental_payment

- Table: payments
 - id (BIGINT, Primary Key, Auto-Increment)

- **booking_id** (BIGINT, maps to bookings.id)
- **tenant_id** (BIGINT, maps to users.id)
- **amount** (DECIMAL(10, 2))
 - **payment_method** (VARCHAR(20), values: CARD, NET_BANKING, UPI, WALLET)
- **status** (VARCHAR(20), values: PENDING, COMPLETED, FAILED, REFUNDED)
- **transaction_reference** (VARCHAR(100))

3. Security & JWT Authentication Protocol

LuxeFlats uses stateless JWT (JSON Web Token) authentication to verify identity across the cluster.

How the Hashing and Verification Flow Works:

1. Login request: The client POSTs JSON credentials to `/api/auth/login` (via Gateway).`
2. Service Hashing: The Auth Service fetches the User record from the Database, compares the raw password against the stored hash using `BCryptPasswordEncoder`, and if valid, generates a JWT.
3. JWT Claims: The JWT is signed with the Shared Secret (`jwt.secret``). The payload contains claims:
 - Subject (`'sub'`): Username
 - Role (`'role'`): `ROLE_TENANT`, `ROLE_OWNER`, or `ROLE_ADMIN`
 - UserID (`'userId'`): Internal database ID of the user
4. Client storage: The React client stores this token in `localStorage` alongside user profile details.`
5. Downstream routing: On subsequent requests, the frontend adds ``Authorization: Bearer <token>` to headers. API Gateway routes the request to target service.`
6. Downstream Verification: The target service executes ``JwtAuthenticationFilter` on every request. It parses the Authorization header, extracts the token, and uses its own `JwtUtil` (configured with the same secret) to decrypt and verify the token signature. If valid, it instantiates a Spring Security UsernamePasswordAuthenticationToken and sets it in the SecurityContextHolder.`

Security Configuration (`SecurityConfig.java`):

- Stateless Session: `SessionCreationPolicy.STATELESS` disables `JSESSIONID` cookie generation.
- CORS mapping: API Gateway acts as the CORS broker, allowing requests from origin `http://localhost:5173/5174`.
- Actuator Endpoints: HTTP requests to `/actuator/**` are bypass-permitted to allow health checks.`

4. Core Backend Microservices & Key Classes

The microservices share a standard layered architecture (Controller -> Service -> Repository -> Database).

Important Classes per Microservice:

1. auth-service:

- **AuthController.java:** Declares REST mapping `/api/auth/register` (POST) and /api/auth/login` (POST).`
- **User.java:** Plain Java entity representation mapping fields to SQL columns.
- **UserRepository.java:** Interface extending `JpaRepository<User, Long>` with helper `findByUsername()`.
- **JwtUtil.java:** Configured with `jwt.secret` and `jwt.expiration-ms` to sign and decrypt claims.

2. property-service:

- **PropertyController.java:** Exposes property registration, search, and availability patching.
- **Property.java:** Entity class representing listings with fields `city`, `rentAmount`, `locality`, and `furnishing`.
- **DataInitializer.java:** Implements `CommandLineRunner` to seed the database with listings on startup.

3. booking-service:

- **BookingController.java:** Exposes status update (`PATCH /api/bookings/{id}/status?status=``) and listings creation.
- **Booking.java:** Lease details entity with check-in/check-out dates (`startDate`/`endDate``) and `BookingStatus``.
- **BookingRequest.java:** Request payload class with `@FutureOrPresent` constraints on check-in dates.

4. payment-service:

- **PaymentController.java:** Declares endpoints to log payments and patch invoice statuses.
- **Payment.java:** Entity logging tenant transactions with a `transactionReference` column.

5. api-gateway:

- **RoutePredicateHandlerMapping & FilteringWebHandler:** Native Spring Cloud Gateway components routing incoming HTTP paths (like `/api/bookings/**``) to downstream ports (like `http://localhost:8083``).

5. End-to-End Booking Request Lifecycle

A tenant's request follows a structured lifecycle across services, transitioning through state machines:

Step 1: The Request Submission

- A Tenant browsing property details clicks 'Book Stay' and selects move-in/move-out dates on the React calendar.
- Frontend dispatches a POST request to `/api/bookings`` via the API Gateway.
- Headers include Authorization Bearer token (JWT) and X-User-Id containing Tenant user.id.
- The booking-service validates check-in dates in `BookingRequest.java`` using `@FutureOrPresent`` annotations, verifying that the move-out date is strictly after the move-in date.
- Upon successful validation, the service instantiates a `Booking`` record in the database with status `PENDING``.

Step 2: Owner Review & Approval

- The Owner logs into their dashboard, which polls `/bookings/property/{id}`` for all properties they own.
- The owner dashboard renders pending stay requests.
- Clicking 'Approve' triggers a request: `PATCH /api/bookings/{id}/status?status=CONFIRMED`` via request parameters.
- The booking-service processes the PATCH request, sets the booking status column to `CONFIRMED`` in the MySQL table, and saves the modified entity.

Step 3: Lease Status Transitions

The booking moves through four distinct states:

- **PENDING:** Initial stay request from tenant, awaiting owner approval.
- **CONFIRMED:** Approved by the owner, awaiting tenant payment.
- **COMPLETED:** Lease finalized after successful payment transaction.
- **CANCELLED:** Dismissed by owner, tenant, or due to payment timeout.

6. Payment Integration & Stay Completion Sync

Once a booking is approved (`CONFIRMED`), the payment flow completes the staying lease.

Step 1: Mock Payment Flow Initiation

- The Tenant navigates to the 'Payments' page on their dashboard. The payment page parses the booking details.
- The user chooses a payment method (e.g. UPI, CARD) and submits the mock payment.

Step 2: Sequential API Synchronization Calls

Since the microservices run in isolated containers, completing a transaction requires the React client to coordinate three sequential requests:

1. Create Payment Log (POST `/api/payments`):

- Creates a transaction record with status `PENDING`, specifying the `bookingId`, `tenantId`, and total rent amount.

2. Complete Payment (PATCH `/api/payments/{id}/status?status=COMPLETED`):

- Simulates invoice clearance, marking the payment transaction record as `COMPLETED` in `flat_rental_payment` database.
- Generates a mock transaction reference (e.g., `TXN-172372834`).

3. Complete Booking (PATCH `/api/bookings/{id}/status?status=COMPLETED`):

- Marks the lease stay as `COMPLETED` in `flat_rental_booking` database. This completes the renting journey.

Data Integrity Considerations:

- In a production system, this sequence is wrapped in a saga pattern or coordinated using message brokers (like RabbitMQ/Kafka) to ensure transactional reliability across service boundaries. In this project, the client orchestrates the calls sequentially, handling fallback logic if a step fails.

7. React Frontend Architecture & Axios Interceptors

The React frontend compiles using Vite. It implements dynamic dashboard rendering, glassmorphic styling using CSS custom variables, and handles E2E API gateway requests.

Key Frontend Modules:

1. axiosConfig.js:

- Establishes a global Axios client pointing to the API Gateway at `http://localhost:8080/api`.
- Request Interceptor: Automatically fetches the stored `token` from `localStorage` and appends `Authorization: Bearer <token>` to request headers. It also extracts the logged-in user details and appends the `X-User-Id` header (mapping to `user.id`).
- Response Interceptor: Listens for standard response codes. If a request returns `401 Unauthorized` (indicating an expired or invalid token), the interceptor automatically clears the user storage and redirects to `/login`.

2. AuthContext.jsx:

- Context provider tracking login states and exposing `login()`, `logout()`, and `user` state globally.
- The login response populates `user.id` and `user.userId` to ensure compatibility across client dashboard pages.

3. Properties.jsx / PropertyDetails.jsx:

- Properties.jsx: Performs client-side sorting and filtering of available flats based on city, bedrooms, furnishing, and monthly rent budget.
- PropertyDetails.jsx: Exposes description lists, image gallery navigation, and conditional CTA buttons (e.g., 'Book Now' for Tenant, 'Edit' for Property Owner, 'Login to Book' for anonymous users).

8. Dynamic Owner Earnings & Dashboard Mechanics

The Owner Dashboard (`OwnerDashboard.jsx`) coordinates data from multiple services to build a unified view without sharing databases at the backend level:

Data Synchronization Flow in Owner Dashboard:

1. **Fetch Properties:** OwnerDashboard dispatches a request to `/api/properties/owner/{ownerId}` to retrieve all listings owned by the logged-in user.
2. **Parallel Bookings Lookup:** For each retrieved property, the client spawns parallel fetch promises to get `/api/bookings/property/{propertyId}` from the booking service. This is mapped to retrieve stay requests.
3. **Fetch Payments:** The dashboard calls `/api/payments` to retrieve payment logs.
4. **Filter & Match Logic:** The frontend performs an inner join in the client code:
 - Creates a set of all booking IDs associated with the owner's properties (`ownerBookingIds`).
 - Filters the payment logs to include only transactions matching those booking IDs.
 - Filters by transaction status `COMPLETED`.
5. **Aggregate Earnings:** Computes the sum of completed payment amounts, updating the state to display the owner's earnings dynamically under the 'Total Earnings' card at the top.

Display of Stay Request Lists:

- Display of all stay requests (PENDING, CONFIRMED, COMPLETED, CANCELLED) is kept in the dashboard list.
- The owner sees action buttons ('Approve' / 'Reject') only when the stay request status is strictly `PENDING`.

9. Production Seeding Data & Indian Cities Setup

To ensure realistic testing flows, the project is configured with a comprehensive dataset representing premium properties in major Indian cities. The property-service seeds this data on startup through its `DataInitializer` class. The seeded database details include:

Supported Cities:

- Delhi / Noida / Gurgaon (NCR Region)
- Mumbai / Pune (Maharashtra Region)
- Bangalore / Hyderabad / Chennai / Kolkata (South and East Hubs)

Data Seeding Properties Configuration:

- The seeder initializes 18 distinct properties (2 properties in each of the 9 supported cities).
- Each property is seeded with 7 high-resolution, realistic image URLs from Unsplash, supporting fully functional image galleries with scrollable thumbnails in the frontend UI.
- Realistic parameters are seeded, including furnishing status (Furnished, Semi-Furnished, Unfurnished), area in square feet, security deposit, locality names, and amenities (WiFi, AC, Parking, Gym, Security, Power Backup).

System Logins for Testing:

The database initializes three default accounts with password 'password123' for immediate E2E validation:

1. Tenant Account: username = `tenant`, password = `password123`, role = `ROLE\_TENANT`
2. Owner Account: username = `owner`, password = `password123`, role = `ROLE\_OWNER`
3. Admin Account: username = `admin`, password = `password123`, role = `ROLE\_ADMIN`

10. System Design & Technical Interview Questions

Use these questions to prepare for technical microservices and system design interviews based on this project:

Q1. How does the API Gateway handle requests in this project, and how is CORS configured?

A1. The API Gateway runs on port 8080. It uses Spring Cloud Gateway routes configured in application.yml. It maps request paths (e.g., /api/properties/**) to specific downstream service ports (e.g., port 8082). CORS is configured globally on the Gateway, defining allowed origins (http://localhost:5173/5174/5175), methods (GET, POST, etc.), and headers, preventing the need to configure CORS individually in each downstream service.

Q2. Describe the database topology. What are the pros and cons of this design?

A2. The project uses a 'Database per Service' topology (flat_rental_auth, flat_rental_property, etc.). Pros include loose coupling (services can scale and be modified independently) and data encapsulation. Cons include complex cross-service queries (which require client-side joins or API composition) and distributed transaction management.

Q3. How is user identity validated downstream without checking the database on every request?

A3. Downstream services use stateless JWT validation. The auth-service issues a cryptographically signed JWT containing userId and role claims. Downstream services intercept incoming requests via a custom JwtAuthenticationFilter, extract the bearer token, and validate its signature using the shared secret key. This validates the user's identity statelessly without database lookups.

Q4. What is the lifecycle of a Booking? Detail the database column changes.

A4. A booking starts as PENDING when a Tenant makes a request. The booking table records check-in/check-out dates, tenantId, and propertyId. When the Owner approves, the status is updated to CONFIRMED. Once the Tenant completes the payment flow successfully, the status is updated to COMPLETED. If dismissed, the status changes to CANCELLED.

Q5. How do you handle distributed transaction issues in this setup?

A5. In this educational project, the client coordinates sequential API calls. For production systems, reliable distributed transactions are implemented using the Saga Pattern (orchestrator or choreography-based) and event sourcing with message brokers like Kafka/RabbitMQ to ensure eventual consistency.